

King Fahd University of Petroleum & Minerals

OPERATIONAL TECHNOLOGY FORENSICS

Reconstructing an unauthorised PLC setpoint change from Modbus/TCP traffic and controller logs

SEC 524 – Computer and Network Forensics · Coursework presentation

Student 1 Name · Student 2 Name

November 2026

The forensic problem

An unapproved setpoint change must be reconstructed, not just detected

Suspected event

Holding register 16 in a PLC was changed from 50 to 90 over Modbus/TCP. No change request was approved for that time.

Investigative question

Which command was sent, from where and when? Did the PLC accept it? And where do the limits of attribution lie?

MITRE ATT&CK for ICS T0836 – Modify Parameter

Changing a setpoint is a known adversary technique, but a packet alone does not show intent. A technician, an HMI script or a scheduled task could produce the same bytes.

Why OT forensics is different

Three constraints we do not usually face in IT forensics

1

You cannot just switch it off

A PLC runs a physical process. Imaging it means stopping production, so most evidence is collected live and passively.

2

The protocol has no users

Modbus carries a unit ID, not a user name. A packet tells us an IP and a port, never a person.

3

Safety comes before evidence

Active scans or test writes on a live controller can harm equipment or people. Every action needs approval from the plant owner.

Evidence sources and what they can prove

Ordered from most to least volatile

PLC run-time state	Very high	Current register value only
Network capture (PCAP)	High	Command, register, value, IP, port, time
PLC / controller audit log	Medium	Whether the PLC applied the change
Engineering workstation	Medium	Account and session behind an IP
Historian / HMI trends	Low–medium	Physical effect over time
Firewall, switch, DHCP logs	Low–medium	IP-to-device mapping and path

Key forensic challenges

Each one can change how reliable a conclusion is

Attribution

IP ≠ account ≠ person. NAT, shared engineering logins and spoofing break the chain.

Time

PLC clocks drift, historians sample. Without UTC and measured offsets, events can be put in the wrong order.

Tools and formats

Vendor logs are proprietary. Even Wireshark mislabelled our traffic on a non-standard port.

Anti-forensics

Logs deleted, value written back, clocks changed. Answer: log forwarding, NTP, passive monitoring.

Our OT-adapted forensic framework

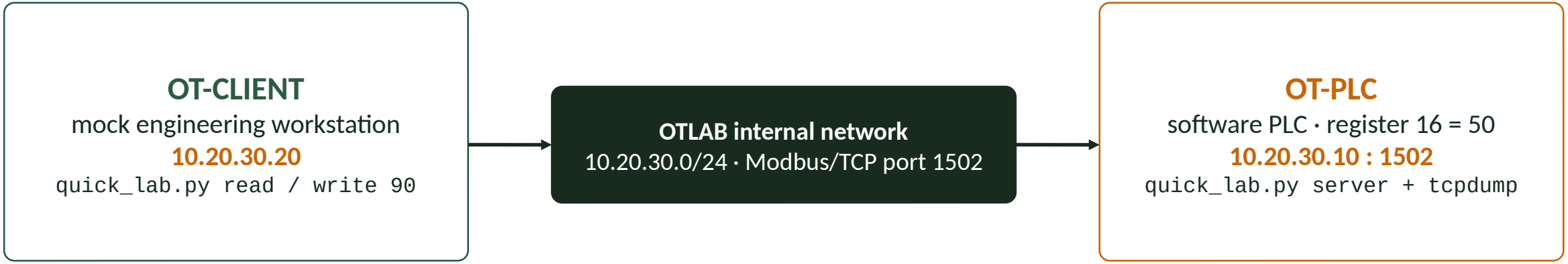
NIST SP 800-86 + OSCAR, extended to 13 stages; orange = changed or added for OT

PREPARE	PRESERVE	ANALYSE	INTERPRET
1 Authorisation + OT safety gate	4 Preservation (stop rotation)	7 Decode + protocol semantics	10 Competing hypotheses H1–H4
2 Identification & scoping	5 Passive-first acquisition	8 Correlation (IP, port, time)	11 Documentation & custody
3 Triage by volatility & process risk	6 SHA-256 integrity checks	9 UTC timeline + clock offsets	12 Fact / inference / unknown

13 Forensic readiness: lessons go back into logging, NTP, register maps and change tickets

Our virtual OT lab

Two Ubuntu VMs in VirtualBox on a private internal network – no real equipment



Baseline (control)

tcpdump → baseline.pcap
3 × read → value 50

Incident

tcpdump → incident.pcap
read 50 → write 90 → read 90

Preserve

stop writers → sha256sum
4 files → SHA256SUMS.txt

Analyse

tshark 4.2.2 + our Python decoder
on verified copies

Evidence collected and preserved

Hashed on OT-PLC, re-verified after transfer: 4 × OK

ID	File	Size	Content	SHA-256 (start)
E1	`baseline.pcap`	2,601 B	30 frames · 16:27:40–16:27:42 UTC	`557fcdac0c507317…`
E2	`incident.pcap`	2,602 B	30 frames · 16:31:17–16:31:33 UTC	`e919a17b09329b85…`
E3	`plc_audit.jsonl`	1,012 B	6 records (5 READ, 1 WRITE_APPLIED)	`fa48dcc0bcd6c439…`
E4	`plc_console.txt`	1,074 B	Console copy of the same 6 records	`c85ff2b1b6bdfe3c…`

Chain of custody: captured on OT-PLC (26 Oct) → hashed with sha256sum → archived as CourseWork.zip → shared with the second team member → extracted and re-verified on the analysis workstation (11 Nov) → analysis only on working copies. A hash proves the copy has not changed since hashing; it does not prove the capture was complete.

The write, byte by byte

incident.pcap frame 14 · 10.20.30.20:45712 → 10.20.30.10:1502 · 16:31:30.220150 UTC

00 01	00 00	00 06	01	06	00 10	00 5A
transaction ID = 1	protocol Modbus	length 6 bytes	unit ID = 1	FC 06 WRITE	register 16	value 90

MBAP header (7 bytes) — *PDU*

Network capture (E2)

Frame 14: FC06 reg 16 = 90
Frame 16: identical echo = accepted

16:31:30.220 UTC

PLC audit log (E3)

WRITE_APPLIED
before 50 → after 90

16:31:30.230 UTC

Read-back (E2)

Frame 26: value 0x005A
Register 16 = 90

16:31:33.944 UTC

Three independent views agree: 10.20.30.20 wrote 90 to register 16 and the PLC accepted it

Timeline and validation

Packet times (E2) vs PLC log times (E3), same incident



Two decoders

tshark and our own Python decoder (raw PCAP bytes) gave the same 12 Modbus PDUs: frames, IPs, ports, registers and values.

Two evidence types

Kernel packet capture and application log agree on all 6 events, their order and values. Baseline: 0 write frames.

Ground truth

We ran the incident ourselves: read → write 90 → read. The reconstruction matches it exactly.

Tool finding: with "Decode As" only, tshark said "Unable to classify as query or response" on port 1502. Setting `-o mbtcp.tcp.port:1502` fixed it. Tools must be validated.

What we can and cannot say

Hypotheses H1–H4, classified as stage 12 of the framework requires

Observed facts

- H1 supported: FC06 write of 90 to register 16 from 10.20.30.20
- H2 supported: PLC echoed it, logged 50 → 90, later read returned 90

Reasonable inferences

- The write came from the OT-CLIENT VM, which held that IP
- H4 rejected: case notes record no approved change, so it was unauthorised

Unknowns

- H3 not supported: which person typed the command; Modbus has no user identity
- Any physical effect: there is no real machine

Bottom line: the evidence identifies a host and an accepted command, not a person.

Limitations, legal context and readiness

What would make a real investigation stronger

Limitations

- Packets and log share one clock: consistent, not independently timed
- Capture on the PLC host, not a SPAN/TAP
- Toy PLC: one register, no vendor firmware
- No workstation artefacts, so no user attribution
- Small dataset (60 frames)

Legal and ethical

- Anti-Cyber Crime Law (M/17): describe the act, leave classification to authorities
- PDPL: employee logs are personal data; minimise and protect
- NCA OTCC-1:2022 applies to in-scope operators
- Safety first: no test writes on live controllers

Readiness

- Passive TAP with rolling, hashed PCAP
- Forward PLC and workstation logs off-box
- One NTP source, log offsets
- Register map + change tickets for every write
- Named accounts + MFA on engineering hosts

Conclusion

1

Reconstructed

Three independent views and two decoders agree, and the result matches the ground truth.

2

Bounded

The evidence proves a host and an accepted command, not a person.

3

Adapted

Safety gate, passive-first collection, protocol semantics and explicit clock handling make OT forensics safe and defensible.

Key references: NIST SP 800-82r3 · SP 800-86 · SP 800-61r3 · ISO/IEC 27037 · Modbus V1.1b3 · MITRE ATT&CK for ICS T0836 · Ahmed et al. 2012 · Awad et al. 2018 · NCA OTCC-1:2022